

DEVFLOW WEBSITE - COMPLETE VIVA PREPARATION GUIDE

Hyper-Detailed Explanation for Every Component

Student Name: Francis Reuben

Level: First-year B.Tech CSE (AI/ML Specialization)

Website Purpose: DevFlow - Product development management and task tracking platform

Duration: 2-hour crash course preparation guide

TABLE OF CONTENTS

1. High-Level Architecture Overview
 2. HTML Structure Breakdown
 3. JavaScript Logic Explained
 4. Key Features & How They Work
 5. Potential Viva Questions & Answers
-

PART 1: HIGH-LEVEL ARCHITECTURE OVERVIEW

What is DevFlow?

DevFlow is a **web-based project management tool** for software developers. It helps users: - Create an account (Sign up/Login) - Define their product ideas - Choose development timelines (1-month, 3-month, 6-month) - Select development pathways (different tech stacks) - Create and manage tasks in a Kanban board (To Do → In Progress → Done) - Manage resources (raw materials, dependencies) - Track product status

System Architecture (3-Tier Model)

PRESENTATION LAYER (HTML)

- Login/Signup Forms
- Kanban Board (To Do/In Progress/Done)
- Task Creation Interface
- Resource Management

↓

LOGIC LAYER (JavaScript)

- Authentication (Login/Signup)
- Task Management

- Data Validation
- Pathway Generation
- Local Storage Management

↓

- DATA LAYER (Browser Local Storage)
- User Credentials (devflow_users)
 - Tasks (devflow_tasks)
 - Resources (devflow_resources)
 - Theme Preference (theme)
 - Current User (devflow_user)

PART 2: HTML STRUCTURE BREAKDOWN

2.1 Document Structure Overview

```
<!DOCTYPE html>
<html>
  <head>
    <!-- Metadata, title, external libraries -->
  </head>
  <body>
    <!-- Auth Page (Login/Signup) -->
    <!-- Main App Page (Dashboard, Tasks, Resources) -->
  </body>
</html>
```

2.2 Head Section (Meta Information)

What is the <head> section? The <head> contains metadata that doesn't display on the page but tells the browser important information.

```
<meta charset="UTF-8">
```

- **Purpose:** Tells the browser the character encoding is UTF-8 (supports all languages: Hindi, Kannada, English, emojis, etc.)
- **Why needed:** Without this, special characters might not display correctly

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- **Purpose:** Makes the website responsive on mobile devices
- **Explanation:**
 - width=device-width → Website width adapts to screen size
 - initial-scale=1.0 → Starts at 100% zoom (no weird zooming)

External Libraries in Head

```
<!-- Bootstrap CSS -->  
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
```

- **Bootstrap:** Pre-built CSS styling framework (makes things look professional)
- **CDN:** Content Delivery Network (loads from internet, not your computer)

```
<!-- Bootstrap Icons -->  
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css">
```

- **Icons library:** Provides pre-made icons (login icon, logout icon, sun icon for dark mode, etc.)

2.3 Body Structure - Two Main Sections

The body has TWO major display sections that toggle:

SECTION 1: Authentication Page (id="auth-page")

```
<div id="auth-page" class="auth-container">  
  <div id="login-form">  
    <input id="login-email" type="email" placeholder="Email">  
    <input id="login-pass" type="password" placeholder="Password">  
    <button onclick="handleAuth(event, 'login')">LOGIN</button>  
  </div>  
  
  <div id="signup-form" style="display: none;">  
    <input id="signup-name" type="text" placeholder="Full Name">  
    <input id="signup-email" type="email" placeholder="Email">  
    <input id="signup-pass" type="password" placeholder="Password">  
    <button onclick="handleAuth(event, 'signup')">SIGN UP</button>  
  </div>  
</div>
```

How it works: 1. When user opens website → Auth page shows (Login tab visible) 2. User clicks “Sign Up” → JavaScript hides login form, shows signup form 3. After successful login/signup → JavaScript hides entire auth-page 4. Main app becomes visible

Key HTML Attributes: - id="login-email" → JavaScript finds this element using `document.getElementById('login-email')` - type="email" → Browser validates that input looks like email - type="password" → Hides typed characters (shows dots instead) - onclick="handleAuth(event, 'login')" → When button clicked, calls JavaScript function

SECTION 2: Main Application Page This section is hidden initially (style="display: none") and shown after login.

2.3.1 Navigation & Header

```
<nav class="navbar">
  <h1 class="navbar-brand">DEVFLOW</h1>
  <button onclick="handleLogout()">LOGOUT</button>
  <button id="theme-toggle" onclick="toggleTheme()">
    <i id="theme-icon" class="bi bi-moon"></i>
  </button>
</nav>
```

Components: - **Navbar:** Navigation bar at the top - **DEVFLOW logo:** Website title - **Logout button:** Calls handleLogout() function - **Theme toggle:** Dark/Light mode switcher - <i class="bi bi-moon"></i> → Moon icon for light mode - Changes to sun icon in dark mode

2.3.2 Idea Input & Pathway Generation Section

```
<div class="idea-section">
  <h2>What are you building?</h2>
  <p>DEFINE YOUR PRODUCT. WE BUILD THE ROADMAP.</p>

  <input id="ideaInput" type="text" placeholder="Your Product Idea">

  <select id="timeframeInput">
    <option>1 Month (Hackathon)</option>
    <option>3 Months (MVP)</option>
    <option>6 Months (Full Launch)</option>
  </select>

  <select id="typeInput">
    <option>Web Application</option>
    <option>Mobile App</option>
    <option>Hardware / IoT</option>
  </select>

  <button onclick="generatePathways()">GENERATE PATHWAYS</button>
</div>
```

How each element works:

HTML Element	Purpose	JavaScript Connection
<code><input id="ideaInput"></code>	User types their product idea	JS reads with <code>document.getElementById('ideaInput').value</code>
<code><select id="timeframeInput"></code>	Dropdown to select timeline	JS reads selected value in <code>generatePathways()</code>
<code><select id="typeInput"></code>	Dropdown for product type	(Used for filtering pathways)
Button with <code>onclick="generatePathways()"</code>	Triggers pathway generation	Calls the <code>generatePathways()</code> function

2.3.3 Pathway Display Section

```
<div id="pathway-cards" class="pathway-container">
  <!-- JavaScript dynamically adds cards here -->
</div>
```

Important: This `<div>` is EMPTY initially. JavaScript fills it with content:

```
document.getElementById('pathway-cards').innerHTML = paths.map(...)
```

The `innerHTML` property allows JavaScript to insert HTML code as text. Example:

```
document.getElementById('pathway-cards').innerHTML = `
  <div class="card">
    <h3>THE RAPID PROTOTYPE</h3>
    <p>React + Firebase</p>
  </div>
`;
```

2.3.4 Kanban Board Section

```
<div class="kanban-board">
  <div class="column" id="todo-column">
    <h3>TO DO</h3>
    <span id="todo-count">0</span>
    <div id="todo-tasks" class="tasks-container">
      <!-- Tasks dynamically added here -->
    </div>
  </div>

  <div class="column" id="inprogress-column">
    <h3>IN PROGRESS</h3>
```

```

    <span id="inprogress-count">0</span>
    <div id="inprogress-tasks" class="tasks-container">
      <!-- Tasks dynamically added here -->
    </div>
  </div>

  <div class="column" id="done-column">
    <h3>DONE</h3>
    <span id="done-count">0</span>
    <div id="done-tasks" class="tasks-container">
      <!-- Tasks dynamically added here -->
    </div>
  </div>
</div>

```

What is a Kanban Board? - Visual task management system with 3 columns
 - Tasks move left-to-right as work progresses - **TO DO:** Not started yet - **IN PROGRESS:** Currently being worked on - **DONE:** Completed

How JavaScript populates it: 1. JavaScript loops through all tasks stored in memory 2. For each task, it finds the correct column based on task status 3. Creates a task card HTML element 4. Inserts it into the correct column div

2.3.5 Task Creation Form

```

<div class="task-creation-form">
  <h3>CREATE TASK</h3>

  <input id="taskName" type="text" placeholder="ITEM NAME">

  <select id="taskSource">
    <option>Core Feature</option>
    <option>UI/Design</option>
    <option>Bug Fix</option>
  </select>

  <select id="taskStatus">
    <option>To Do</option>
    <option>In Progress</option>
    <option>Done</option>
  </select>

  <button onclick="createTask()">CREATE TASK</button>
</div>

```

User Flow: 1. User types task name 2. Selects task category (Core Fea-

ture/UI/Design/Bug Fix) 3. Selects initial status 4. Clicks “CREATE TASK”
→ JavaScript: - Creates task object - Saves to localStorage - Refreshes Kanban board display

2.3.6 Resources Management Section

```
<div class="resources-section">
  <h3>Manage Resources</h3>
  <table>
    <thead>
      <tr>
        <th>RESOURCE NAME</th>
        <th>SOURCE / ORIGIN</th>
        <th>STATUS</th>
      </tr>
    </thead>
    <tbody id="resources-list">
      <!-- Resources dynamically added here -->
    </tbody>
  </table>

  <div class="add-resource-form">
    <input id="resourceName" type="text" placeholder="Resource Name">
    <input id="resourceSource" type="text" placeholder="Source/Origin">
    <select id="resourceStatus">
      <option>Planned</option>
      <option>Ordered</option>
      <option>In Stock</option>
    </select>
    <button onclick="addResource()">ADD RESOURCE</button>
  </div>
</div>
```

Purpose: Track materials/dependencies needed for project - **Resources table:** Displays existing resources - **Add form:** User input to create new resources - **Status options:** Track resource availability (not yet planned → ordered → in stock)

PART 3: JAVASCRIPT LOGIC EXPLAINED

3.1 Global Variables (App Memory)

```
let tasks = [];           // Array to store all tasks
let resources = [];       // Array to store all resources
```

```
let currentUser = null; // Currently logged-in user object
```

What are these? - Arrays: Special variable type that holds multiple values in order - **Global variables:** Accessible from all functions - When page refreshes, these reset (that's why we use localStorage!)

Example:

```
tasks = [
  { id: 1, name: "Design Homepage", status: "To Do" },
  { id: 2, name: "Build API", status: "In Progress" }
]
```

3.2 DOMContentLoaded Event (Page Initialization)

```
document.addEventListener('DOMContentLoaded', () => {
  // This code runs when HTML finishes loading

  // Check if user already logged in
  if(localStorage.getItem('devflow_user')) {
    currentUser = JSON.parse(localStorage.getItem('devflow_user'));
    document.getElementById('auth-page').style.display = 'none'; // Hide login
    loadData(); // Load user's tasks and resources
  }

  // Check for dark mode preference
  if (localStorage.getItem('theme') === 'dark') {
    document.documentElement.setAttribute('data-theme', 'dark');
    document.getElementById('theme-icon').className = 'bi bi-sun me-3';
  }

  renderResources();
});
```

Why is this needed? - HTML loads line-by-line - If JavaScript runs BEFORE HTML loads, it can't find elements (errors!) - DOMContentLoaded waits for HTML to fully load, THEN runs code

What happens: 1. **Page opens** → HTML loads → DOMContentLoaded triggers 2. **Check localStorage** → Has user data? (means user logged in before) 3. If yes → Hide login page, show dashboard 4. **Check dark mode** → Apply if user preferences say so 5. **Load user data** → Fetch tasks/resources from storage

3.3 Authentication System

3.3.1 Toggle Between Login & Signup Forms

```
function toggleAuth(type) {
  const isLogin = type === 'login';

  document.getElementById('login-form').style.display = isLogin ? 'block' : 'none';
  document.getElementById('signup-form').style.display = isLogin ? 'none' : 'block';

  document.getElementById('login-tab').classList.toggle('active', isLogin);
  document.getElementById('signup-tab').classList.toggle('active', !isLogin);
}
```

Ternary Operator Explanation (condition ? true_value : false_value):

```
const isLogin = type === 'login';
// If type is 'login', isLogin = true
// Otherwise, isLogin = false
```

Example execution:

User clicks "LOGIN" tab
→ toggleAuth('login') called
→ isLogin = true
→ login-form display = 'block' (shows)
→ signup-form display = 'none' (hides)

3.3.2 Handle Login & Signup

```
function handleAuth(e, type) {
  e.preventDefault(); // Prevent page reload (default form behavior)

  const db = JSON.parse(localStorage.getItem('devflow_users') || '[]');
  // Get all users from storage, or empty array if no data

  if (type === 'login') {
    // ===== LOGIN LOGIC =====
    const email = document.getElementById('login-email').value;
    const pass = document.getElementById('login-pass').value;

    // Find user with matching email AND password
    const user = db.find(u => u.email === email && u.pass === pass);

    if (user) {
      // Login successful
      currentUser = user;
    }
  }
}
```

```

        localStorage.setItem('devflow_user', JSON.stringify(user));
        completeLogin();
    } else {
        // Login failed
        showToast('Invalid Email or Password', 'danger');
    }
}
else {
    // ===== SIGNUP LOGIC =====
    const name = document.getElementById('signup-name').value;
    const email = document.getElementById('signup-email').value;
    const pass = document.getElementById('signup-pass').value;

    // Check if email already exists
    if(db.some(u => u.email === email)) {
        showToast('User already exists', 'danger');
        return;
    }

    // Create new user object
    const newUser = { name, email, pass };

    // Add to user database
    db.push(newUser);

    // Save updated database back to localStorage
    localStorage.setItem('devflow_users', JSON.stringify(db));

    // Set as current user and complete login
    currentUser = newUser;
    localStorage.setItem('devflow_user', JSON.stringify(newUser));
    completeLogin();
}
}

```

Key Methods Explained:

Method	Purpose	Example
.find()	Find FIRST item matching condition	db.find(u => u.email === email)
.some()	Check if ANY item matches condition (true/false)	db.some(u => u.email === email)
.push()	Add item to end of array	db.push(newUser)

Method	Purpose	Example
JSON.parse()	Convert text to JavaScript object	JSON.parse('{ "name": "John" }')
JSON.stringify()	Convert object to text for storage	JSON.stringify({name: "John"})

Flow Diagram:

```

User submits form
↓
handleAuth() called
↓
  LOGIN: Find matching user in database
        Found? → Save to storage + completeLogin()
        Not found? → Show error toast

  SIGNUP: Create new user
        Email exists? → Show error
        Email new? → Create object + save to storage + completeLogin()

```

3.3.3 Complete Login Process

```

function completeLogin() {
  // Fade out auth page
  document.getElementById('auth-page').style.opacity = '0';

  // After 300ms animation, hide completely and show app
  setTimeout(() => {
    document.getElementById('auth-page').style.display = 'none';
    loadData(); // Load user's tasks and resources
  }, 300);

  showToast('Welcome to the Workspace', 'success');
}

```

What is setTimeout? - Delays code execution by specified milliseconds - Here: Waits 300ms for fade-out animation to complete - Then hides page and loads data

3.3.4 Logout Function

```

function handleLogout() {
  localStorage.removeItem('devflow_user'); // Remove current user data
}

```

```
location.reload(); // Refresh page (shows login screen again)
}
```

3.4 Data Loading

```
function loadData() {
  // Load tasks from storage, or empty array if no tasks
  tasks = JSON.parse(localStorage.getItem('devflow_tasks') || '[]');

  // Load resources from storage, or use default sample resources
  resources = JSON.parse(localStorage.getItem('devflow_resources') || JSON.stringify([
    { id: 'r1', name: 'Leather Raw Material', source: 'Kanpur, UP', status: 'In Stock' },
    { id: 'r2', name: 'Silk Fabrics', source: 'Mysore, Karnataka', status: 'Ordered' }
  ]));

  renderBoard(); // Display tasks on Kanban board
  renderResources(); // Display resources in table
}
```

Why do we need this? - localStorage stores data even after page closes - On next login, we retrieve saved data - Default resources provided if user has no saved resources yet

3.5 Pathway Generation

```
function generatePathways() {
  const idea = document.getElementById('ideaInput').value;

  // Validate: User must enter an idea
  if(!idea) return showToast("Enter a product idea first!", "danger");

  const timeframe = document.getElementById('timeframeInput').value;

  // Define two development pathways
  const paths = [
    {
      title: "THE RAPID PROTOTYPE",
      desc: `Ideal for a ${timeframe}-month timeline. Focuses on MVP.`,
      stack: "React + Firebase",
      tasks: ["Setup Repo", "Design Wireframes", "Auth (OTP)", "Integrate UPI", "Build MVP"],
    },
    {
      title: "THE SCALABLE FOUNDATION",
      desc: "Built for scaling across Karnataka.",
    }
  ]
}
```

```

      stack: "MERN / AWS",
      tasks: ["DB Schema", "Docker & CI/CD", "Design System", "Backend APIs", "Payment Gatew
    }
  ];

  // Generate HTML cards and insert into page
  document.getElementById('pathway-cards').innerHTML = paths.map((p, i) => `
    <div class="pathway-card">
      <h3>${p.title}</h3>
      <p>${p.desc}</p>
      <p><strong>Tech Stack:</strong> ${p.stack}</p>
      <button onclick="selectPathway(${i})">SELECT THIS PATH</button>
    </div>
  `).join('');
}

```

Advanced JavaScript: .map() Function

.map() transforms each item in array:

```

paths.map((p, i) => `<div>${p.title}</div>`)
// For each path object 'p' at index 'i'
// Return HTML string with that path's title
// Result: Array of HTML strings

```

Then .join('') combines them:

```

['<div>Path 1</div>', '<div>Path 2</div>'].join('')
// Result: '<div>Path 1</div><div>Path 2</div>'

```

Template Literals (Backticks):

```

`Hello ${name}, you have ${count} tasks`
// ${} inserts JavaScript variable values
// Much cleaner than: "Hello " + name + ", you have " + count + " tasks"

```

3.6 Task Management

3.6.1 Create Task

```

function createTask() {
  const name = document.getElementById('taskName').value;
  const source = document.getElementById('taskSource').value;
  const status = document.getElementById('taskStatus').value;

  // Validation
  if(!name) return showToast('Enter task name!', 'danger');
}

```

```

// Create task object
const task = {
  id: Date.now(), // Unique ID using current timestamp
  name,
  source,
  status
};

// Add to tasks array
tasks.push(task);

// Save to localStorage
localStorage.setItem('devflow_tasks', JSON.stringify(tasks));

// Refresh display
renderBoard();

// Clear form
document.getElementById('taskName').value = '';

showToast('Task created!', 'success');
}

```

Why use `Date.now()` as ID? - Returns milliseconds since Jan 1, 1970 - Always unique (never repeats) - Prevents duplicate task IDs

3.6.2 Render Kanban Board

```

function renderBoard() {
  // Get all status types
  const statuses = ['To Do', 'In Progress', 'Done'];

  // For each status
  statuses.forEach(status => {
    // Find all tasks with this status
    const statusTasks = tasks.filter(t => t.status === status);

    // Update count
    document.getElementById(`${status.toLowerCase().replace(' ', '')}-count`).textContent =

    // Generate HTML for tasks
    const tasksHTML = statusTasks.map(t => `
      <div class="task-card">
        <h5>${t.name}</h5>
        <small>${t.source}</small>
      </div>
    `);
  });
}

```

```

        <button onclick="deleteTask(${t.id})">Delete</button>
        <button onclick="moveTask(${t.id}, 'next')">→</button>
    </div>
`}).join('');

    // Insert into column
    document.getElementById(`${status.toLowerCase().replace(' ', '')}-tasks`).innerHTML = tasks
  });
}

```

How it works:

1. **For each column (To Do, In Progress, Done):**
2. **Filter tasks** → Keep only tasks with that status
3. **Update count** → Show how many tasks in column
4. **Generate HTML** → Create task cards
5. **Insert into page** → Update column div

String manipulation:

```

'In Progress'.toLowerCase().replace(' ', '')
// 'In Progress' → 'in progress' → 'inprogress'
// Matches HTML id: inprogress-tasks

```

3.6.3 Move Task Between Columns

```

function moveTask(taskId, direction) {
    // Find the task
    const task = tasks.find(t => t.id === taskId);
    if(!task) return;

    // Define status flow
    const statusFlow = ['To Do', 'In Progress', 'Done'];
    const currentIndex = statusFlow.indexOf(task.status);

    if(direction === 'next' && currentIndex < 2) {
        // Move to next status (left to right)
        task.status = statusFlow[currentIndex + 1];
    } else if(direction === 'prev' && currentIndex > 0) {
        // Move to previous status (right to left)
        task.status = statusFlow[currentIndex - 1];
    }

    // Save and refresh
    localStorage.setItem('devflow_tasks', JSON.stringify(tasks));
    renderBoard();
}

```

```
}
```

Visual example:

```
To Do → In Progress → Done
      ↑               ↓
direction='prev'    direction='next'
```

3.6.4 Delete Task

```
function deleteTask(taskId) {
  // Confirm deletion
  if(!confirm('Delete this task?')) return;

  // Remove from array (keep only tasks that DON'T match this ID)
  tasks = tasks.filter(t => t.id !== taskId);

  // Save and refresh
  localStorage.setItem('devflow_tasks', JSON.stringify(tasks));
  renderBoard();
  showToast('Task deleted', 'success');
}
```

.filter() method:

```
[1, 2, 3, 4].filter(n => n > 2) // Result: [3, 4]
// Keep only items where condition is true

tasks.filter(t => t.id !== taskId)
// Keep only tasks whose ID is NOT equal to taskId
// Effectively removes the task
```

3.7 Resource Management

```
function addResource() {
  const name = document.getElementById('resourceName').value;
  const source = document.getElementById('resourceSource').value;
  const status = document.getElementById('resourceStatus').value;

  if(!name || !source) return showToast('Fill all fields!', 'danger');

  // Create resource object
  const resource = {
    id: 'r' + Date.now(),
    name,
```



```

        source,
        status
    };

    // Add and save
    resources.push(resource);
    localStorage.setItem('devflow_resources', JSON.stringify(resources));

    renderResources();

    // Clear form
    document.getElementById('resourceName').value = '';
    document.getElementById('resourceSource').value = '';

    showToast('Resource added!', 'success');
}

function renderResources() {
    const html = resources.map(r => `
        <tr>
            <td>${r.name}</td>
            <td>${r.source}</td>
            <td>
                <span class="status-badge">${r.status}</span>
                <button onclick="deleteResource('${r.id}')">Remove</button>
            </td>
        </tr>
    `).join('');

    document.getElementById('resources-list').innerHTML = html;
}

function deleteResource(resourceId) {
    resources = resources.filter(r => r.id !== resourceId);
    localStorage.setItem('devflow_resources', JSON.stringify(resources));
    renderResources();
}

```

Pattern: Add → Save → Render (same pattern as tasks)

3.8 Theme Toggle

```

function toggleTheme() {
    const isDark = document.documentElement.getAttribute('data-theme') === 'dark';

```

```

if(isDark) {
  // Switch to light mode
  document.documentElement.removeAttribute('data-theme');
  document.getElementById('theme-icon').className = 'bi bi-moon me-3';
  localStorage.setItem('theme', 'light');
} else {
  // Switch to dark mode
  document.documentElement.setAttribute('data-theme', 'dark');
  document.getElementById('theme-icon').className = 'bi bi-sun me-3';
  localStorage.setItem('theme', 'dark');
}
}

```

How it works: 1. Check current theme 2. Toggle opposite theme 3. Update icon 4. Save preference

3.9 Toast Notifications

```

function showToast(message, type) {
  // type can be 'success', 'danger', 'info'
  const toast = document.createElement('div');
  toast.className = `toast alert-${type}`;
  toast.textContent = message;
  toast.style.cssText = `
    position: fixed;
    bottom: 20px;
    right: 20px;
    padding: 15px 20px;
    border-radius: 5px;
    color: white;
    z-index: 1000;
  `;

  document.body.appendChild(toast);

  // Remove after 3 seconds
  setTimeout(() => toast.remove(), 3000);
}

```

What does it do? - Creates popup message at bottom-right - Auto-disappears after 3 seconds - Shows success/error/info messages

PART 4: KEY FEATURES & HOW THEY WORK

Feature 1: Authentication System

Components: - Login form (email + password) - Signup form (name + email + password) - User database (localStorage) - Session management (currentUser)

Flow:

User → Forms → `handleAuth()` → Validate → Store → `completeLogin()` → Show Dashboard

Security Note: This is for practice only! Real apps use: - Backend server (not browser storage) - Password hashing (not plain text) - Encrypted HTTPS

Feature 2: Kanban Board

Components: - 3 columns (To Do, In Progress, Done) - Task cards (draggable in real apps) - Count badges - Move/Delete buttons

Data Structure:

```
{
  id: 1234567890,
  name: "Design Homepage",
  source: "UI/Design",
  status: "In Progress"
}
```

Feature 3: Pathway Generation

Purpose: Suggest development approaches based on project type

Example Output:

RAPID PROTOTYPE (1-3 months)

Tech Stack: React + Firebase

Tasks: 6 steps

Best for: Hackathons, MVPs

SCALABLE FOUNDATION (3-6 months)

Tech Stack: MERN + AWS

Tasks: 6 steps

Best for: Production apps

Feature 4: Resource Tracking

Purpose: Manage project dependencies and materials

Example: | Resource | Source | Status | |—|—|—| | Leather Material | Kanpur, UP | In Stock | | APIs | Third-party | Ordered |

PART 5: POTENTIAL VIVA QUESTIONS & ANSWERS

Q1: What does `DOMContentLoaded` do?

Answer: It's an event that fires when the entire HTML document has finished loading. We use it because: - JavaScript can't access HTML elements before they're loaded - If we run code before HTML loads, we get "element not found" errors - `DOMContentLoaded` waits for HTML → then runs our initialization code (checking login status, loading themes, etc.)

Code: `document.addEventListener('DOMContentLoaded', () => { /* code */ })`

Q2: What's the difference between `localStorage` and regular variables?

Answer:

Regular Variables	localStorage
Lost when page refreshes	Survives page refresh
Only in RAM	Stored on disk
Fast but temporary	Slower but permanent
Used: During session	Used: Across sessions

Example:

```
let user = "John"; // Lost after refresh
localStorage.setItem('user', 'John'); // Survives refresh
```

Q3: How does the login system work?

Answer: 1. User enters email and password 2. `handleAuth()` retrieves all users from `localStorage` 3. Uses `.find()` to search for matching user 4. If found → save current user + hide login page + load dashboard 5. If not found → show error message

Code concept:

```
const user = db.find(u => u.email === email && u.pass === pass);
// Finds first user where email AND password match
```

Q4: What is .map() and how is it used in pathway generation?

Answer: .map() transforms each item in an array using a function:

```
const pathways = [
  { title: "RAPID", stack: "React" },
  { title: "SCALABLE", stack: "MERN" }
];

const html = pathways.map(p => `<h3>${p.title}</h3>`);
// Result: ['<h3>RAPID</h3>', '<h3>SCALABLE</h3>']
```

Then .join('') combines them:

```
html.join('') // Result: '<h3>RAPID</h3><h3>SCALABLE</h3>'
```

In the code:

```
paths.map((p, i) => `<div>${p.title}</div>`).join('')
// Creates HTML for each pathway and joins into single string
```

Q5: How does the Kanban board update when a task status changes?

Answer: 1. User clicks move button → moveTask() function called 2. Function finds task by ID 3. Changes status to next in sequence (To Do → In Progress → Done) 4. Saves to localStorage 5. Calls renderBoard() to update display 6. renderBoard() filters tasks by status and recreates column content

Code:

```
const statusFlow = ['To Do', 'In Progress', 'Done'];
task.status = statusFlow[currentIndex + 1]; // Next status
```

Q6: What's the purpose of e.preventDefault() in handleAuth()?

Answer: When a form is submitted, the browser automatically: - Reloads the page - Sends data to a server (default behavior)

e.preventDefault() stops this behavior so we can: - Handle login with JavaScript - Check credentials manually - Update page without reloading

Without it: Page would reload and lose our currentUser variable

Q7: How is the Kanban board organized in HTML?

Answer: Three columns are created as separate `<div>` elements:

```
<div id="todo-column">
  <div id="todo-tasks" class="tasks-container">
    <!-- Tasks inserted here by JavaScript -->
  </div>
</div>
```

JavaScript: 1. Groups tasks by status using `.filter()` 2. For each group, generates HTML task cards 3. Inserts HTML into corresponding column div via `.innerHTML`

Q8: What happens when you click “LOGOUT”?

Answer: 1. `handleLogout()` is called 2. Removes `devflow_user` from local-Storage 3. Calls `location.reload()` to refresh page 4. `DOMContentLoaded` checks for user → none found 5. Auth page is shown (Login screen)

Code:

```
localStorage.removeItem('devflow_user'); // Clear session
location.reload(); // Refresh page
```

Q9: How does the theme toggle work?

Answer: 1. Check current theme using `getAttribute('data-theme')` 2. If dark mode → remove it (switch to light) 3. If light mode → set to dark 4. Update icon (moon/sun) 5. Save preference to `localStorage`

Result: Theme preference survives page refresh

Q10: What’s the data flow for creating a new task?

Answer:

```
User Input
↓
createTask() function
↓
Create task object with unique ID
↓
Add to tasks array
↓
```

Save to localStorage: `JSON.stringify(tasks)`
↓
Call `renderBoard()`
↓
`renderBoard()` filters tasks and recreates HTML
↓
Updated Kanban board shown to user

Key insight: Create → Save → Render is the pattern for all data operations

Q11: What is `JSON.stringify()` and `JSON.parse()`?

Answer:

`JSON.stringify()` - Convert object to text for storage:

```
const user = { name: "John", email: "john@email.com" };
const text = JSON.stringify(user);
// Result: '{"name":"John","email":"john@email.com"}'
// This text can be stored in localStorage
```

`JSON.parse()` - Convert text back to object:

```
const text = '{"name":"John","email":"john@email.com"}';
const user = JSON.parse(text);
// Result: { name: "John", email: "john@email.com" }
// Now we can access: user.name, user.email
```

Why needed: localStorage only stores text, not objects!

Q12: How does the signup validation work?

Answer:

```
if(db.some(u => u.email === email))
  return showToast('User already exists', 'danger');
```

.some() method: - Returns true if ANY item matches condition - Returns false if NO items match

Example:

```
const db = [
  { email: "john@email.com" },
  { email: "jane@email.com" }
];
```

```
db.some(u => u.email === "john@email.com") // true (found)
db.some(u => u.email === "bob@email.com") // false (not found)
```

This prevents duplicate emails in the database.

Q13: What's the purpose of `id: Date.now()` when creating tasks?

Answer: `Date.now()` returns milliseconds since January 1, 1970:

```
Date.now() // Example: 1704423456789
```

Why use it as ID: - Always unique (increments every millisecond) - No duplicates possible - Easy to generate (no database needed) - Can be used to sort by creation time

Alternative IDs: - Random: `Math.random()` (possible duplicates) - Sequential: Need to track last ID - UUID: More complex but industry standard

Q14: How does the form clearing work after task creation?

Answer:

```
document.getElementById('taskName').value = '';
```

This sets the input's value to empty string, clearing the form.

Why after saving: - User enters task - We save it - Clear form for next task - Better UX (user doesn't have to manually clear)

Q15: What would happen if you try to login with wrong password?

Answer: 1. `handleAuth()` called 2. `.find()` searches for user with matching email AND password 3. No match found → `.find()` returns `undefined` 4. `if(user)` is false 5. Error toast shown: "Invalid Email or Password" 6. User stays on login page

Security note: Real apps shouldn't say "wrong password" (hints hackers that email exists) but this is educational code!

QUICK REFERENCE: COMMON PATTERNS

Pattern 1: Read Input

```
const value = document.getElementById('inputId').value;
```


Pattern 2: Show/Hide Element

```
document.getElementById('elementId').style.display = 'none';    // Hide
document.getElementById('elementId').style.display = 'block';   // Show
```

Pattern 3: Create & Store Object

```
const item = { id: Date.now(), name: "text", status: "value" };
items.push(item);
localStorage.setItem('items', JSON.stringify(items));
```

Pattern 4: Load & Parse

```
const items = JSON.parse(localStorage.getItem('items') || '[]');
```

Pattern 5: Update Display

```
const html = items.map(item => `<div>${item.name}</div>`).join('');
document.getElementById('container').innerHTML = html;
```

Pattern 6: Filter Array

```
const filtered = items.filter(item => item.status === 'active');
```

Pattern 7: Find Single Item

```
const item = items.find(item => item.id === 123);
```

Pattern 8: Delete Item

```
items = items.filter(item => item.id !== 123);
```

FINAL TIPS FOR VIVA

1. **Understand, don't memorize:** Know WHY each line exists
2. **Know the flow:** User action → Function → Update data → Update display
3. **Three core layers:** HTML (structure) → JavaScript (logic) → localStorage (data)
4. **localStorage basics:** Text-based, key-value pairs, survives refresh
5. **DOM manipulation:** `.getElementById()`, `.innerHTML`, `.value`
6. **Array methods:** `.map()`, `.filter()`, `.find()`, `.some()`, `.push()`
7. **Common pattern:** Input → Validate → Create object → Save → Render
8. **Events:** onclick attribute in HTML + function in JavaScript
9. **Async patterns:** `setTimeout()`, animations, callbacks
10. **Error handling:** `showToast()` for user feedback

EXPECTED VIVA DURATION

- **Part 1 (Auth System):** 8-10 minutes
- **Part 2 (Task Management):** 10-12 minutes
- **Part 3 (Resources):** 5-7 minutes
- **Part 4 (General Questions):** 10-15 minutes

Total: 45-60 minutes (with time for follow-ups)

GOOD LUCK!

Remember: Your teacher wants to see if you understand the **concepts**, not memorize every line. Focus on: - How data flows - Why each function exists - How HTML and JavaScript connect - What localStorage does

Final Confidence Tip: You generated this with AI, but understanding it is 100% YOUR achievement. Explain clearly, ask clarifying questions if confused, and show your logical thinking. Good luck, Francis!